PRD: Website Visitor Engagement Analytics

Predicting Purchase Intent from Browsing Behavior

Subject: Data Science | **Version:** 2.0 (complete spec) | **Dataset:** UCI Online Shoppers Purchasing Intention

---

1. Overview

1.1 Problem

Only a small share of e-commerce sessions end in a purchase. Intervening on every visitor (discount popups, retargeting) wastes money. This project builds a session-level classifier that ranks sessions by purchase intent, quantifies how much of its power comes from a purchase-path-tied feature (`PageValues`), and explains which engagement behaviors drive conversion.

1.2 Central Thesis (the storyline every section supports)

> PageValues carries roughly half of the predictive signal but is tied to the purchase path by construction. Without it, a deployable model still gives a real but modest gain over naive baselines, and that gain can be quantified honestly.

1.3 Goals

1. Build a leakage-aware, imbalance-aware purchase-intent classifier.
2. Quantify the PageValues dependence (ablation).
3. Prove the model beats trivial baselines (majority class, one-feature rules, single-feature rankings).
4. Convert scores into a business decision (cost-based threshold, lift table).
5. Explain drivers with consistent, correctly aggregated interpretability.
6. Document limitations honestly.

1.4 Non-Goals

- Causal claims, deep learning, production streaming deployment, AutoML.

1.5 Success Criteria

Criterion	Target
Headline metric	PR-AUC (not accuracy); must beat no-skill (~0.156) and the best single-feature ranking
Rigor	No leakage: all fitting inside CV pipelines; test set touched once
Pre-registration	Primary model, threshold and cost ratio fixed before the test run
Insight	PageValues ablation + corrected SHAP + permutation importance agree or disagreement is reported
Honesty	Limitations section lists every skipped/weak item
Reproducibility	Fresh "restart and run all" succeeds; pinned versions; fixed seeds

---

2. Users and Use Case

- **Primary reader:** examiner/instructor (graded argument).
- **Business persona (framing):** e-commerce growth/marketing team that wants to target interventions at high-intent sessions.
- **Decision supported:** "Which sessions get an intervention?" via score bands and a cost-based threshold.

---

3. Data Specification

3.1 Source

UCI *Online Shoppers Purchasing Intention Dataset* (ucimlrepo id 468, or Kaggle CSV). 12,330 sessions, 17 features + `Revenue` .

3.2 Features

Group	Columns
Page counts	Administrative, Informational, ProductRelated

Group	Columns
Durations (s)	Administrative_Duration, Informational_Duration, ProductRelated_Duration
GA metrics	BounceRates, ExitRates, PageValues
Context	SpecialDay, Month, Weekend
Technical IDs (nominal)	OperatingSystems, Browser, Region, TrafficType
Visitor	VisitorType (New_Visitor / Returning_Visitor / Other)
Target	Revenue (bool)

3.3 Known Data Issues (handle and document each)

Issue	Handling
~125 duplicate rows	Drop before any split; report count
Heavy right skew	log1p on counts/durations/rates
BounceRates vs ExitRates r ≈ 0.90	Keep both for trees/LR (test showed no effect); judge importance as a group
10 of 12 months (Jan, Apr missing)	State as limitation; Month cannot be trusted for unseen months
Feb: ~181 sessions, ~2% conversion	Never interpret as seasonality; flag as too small
"Other" visitor type n≈81	Do not draw conclusions
Technical IDs are nominal codes	Treat as categorical, never ordinal
PageValues tied to purchase path	Mandatory ablation; never call it "proven leakage"
Extreme durations (up to ~7.5 h)	Report count above 3 h; keep (small share) but note
GA metrics are page-level aggregates	Check UCI description; state live use needs past-traffic-only computation

---

4. Scope

4.1 Core (must ship)

EDA, cleaning, preprocessing pipeline, 3 models, light tuning, OOF threshold selection, one-shot test evaluation, PageValues ablation, interpretation, limitations, report, notebook, README.

4.2 Extension tiers (optional; each has acceptance criteria in Section 9)

- **Tier A (cheap, strengthens rigor):** matched-settings model comparison, learning curve, SMOTE-in-CV check, calibration, Month and ID-column ablations, single-feature benchmarks.
- **Tier B (adds product value):** Streamlit scoring app with score bands, k-means segmentation, cost-sensitivity dashboard.
- **Tier C (adds modeling depth):** nested CV for unbiased tuning estimate, repeated CV with paired tests, model stacking/ensemble check, monotonic constraints.
- **Tier D (rebuild, only with spare weeks):** event-level dataset for true early-session prediction.

4.3 Out of Scope

Deep learning, causal inference, A/B test design, real-time infrastructure.

---

5. Design Principles (non-negotiable rules)

1. **Dedup before split.**
2. **Stratified 80/20 split, fixed seed (42);** test set is locked until the primary model, threshold and R are fixed and written down.
3. **All preprocessing inside sklearn** `Pipeline` **/** `ColumnTransformer` **;** any resampling (SMOTE) only inside CV via `imblearn.Pipeline` .
4. **Headline metric = PR-AUC.** Accuracy appears only next to the majority baseline (~0.844).
5. **Thresholds chosen on out-of-fold (OOF) train predictions,** never on test.
6. **Cost ratio R is an assumption;** always show a sensitivity table (e.g., R = 3, 5, 10).
7. **Compare costs against the cheaper naive baseline** (do-nothing costs `R × positives` ; flag-all costs `negatives` ; they cross near R ≈ negatives/positives).
8. **Model scores are not probabilities** (class weighting distorts them); use for ranking or calibrate first.
9. **Report correlated features as groups;** permutation importance underrates them.
10. **SHAP group attribution = sum signed SHAP within a group per session, then take |mean|.** Never sum |SHAP| across one-hot columns (inflates high-cardinality columns).
11. **Bootstrap CIs reflect test-sampling uncertainty only;** label them so.
12. **Associations, not causes.**

---

6. Detailed Implementation Specification

6.1 Environment

```
pip install pandas==2.2.3 numpy==2.1.3 scipy==1.16.3 scikit-learn==1.6.1 \
            xgboost==3.4.1 shap==0.54.1 matplotlib==3.10.0 seaborn==0.13.2
\
            joblib==1.6.0 ucimlrepo imbalanced-learn streamlit
```

Platform: Google Colab (mount Drive to persist) or local venv. Seed `SEED = 42` everywhere.

6.2 Load and Clean

```
from ucimlrepo import fetch_ucirepo
ds = fetch_ucirepo(id=468)
df = pd.concat([ds.data.features, ds.data.targets], axis=1)

df = df.drop_duplicates().reset_index(drop=True)
df["Revenue"] = df["Revenue"].astype(int)
df["Weekend"] = df["Weekend"].astype(int)
for c in ["OperatingSystems","Browser","Region","TrafficType"]:
    df[c] = df[c].astype("category")
# assert rates in [0,1], durations/counts >= 0
```

6.3 EDA (6 to 8 figures, each with a one-line takeaway)

1. Class balance
2. Conversion by month **with counts** (flag Feb as unreliable)
3. Conversion by VisitorType, Weekend, TrafficType (n ≥ 100)
4. log1p KDEs of PageValues, ProductRelated_Duration, BounceRates, ExitRates by outcome
5. Correlation heatmap (annotate BounceRates/ExitRates)
6. PageValues zero vs non-zero conversion (leakage preview)
7. Binned ExitRates and ProductRelated_Duration vs conversion (with n per bin)

6.4 Split

```
X = df.drop(columns="Revenue"); y = df["Revenue"]
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, stratify=y, random_state=SEED)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

6.5 Preprocessing Pipeline

```
skewed = ["Administrative","Administrative_Duration","Informational","Inf
          "ProductRelated","ProductRelated_Duration","BounceRates","ExitR
passthrough = ["SpecialDay","Weekend"]
cats = ["Month","VisitorType","OperatingSystems","Browser","Region","Traf

def make_preprocessor(drop=(), cats_used=cats):
    return ColumnTransformer([
        ("skewed", Pipeline([
            ("log", FunctionTransformer(np.log1p, feature_names_out="one-
            ("scale", StandardScaler())]), [c for c in skewed if c not in
        ("pass", "passthrough", passthrough),
        ("cat", OneHotEncoder(handle_unknown="ignore", min_frequency=30),
    ])
```

Note: `feature_names_out="one-to-one"` lets `get_feature_names_out()` work for SHAP labels. Without it, names must be built manually (skewed, then passthrough, then one-hot names).

6.6 Baselines (train only, nothing fitted unless stated)

Baseline	Purpose
Majority class	Shows why accuracy misleads (~0.844)
Rule `PageValues > 0`	The bar the with-PageValues model must beat
Single-feature rankings (ExitRates inverted, ProductRelated_Duration, ProductRelated) via PR-AUC	The bar the deployable model must beat
Logistic regression (balanced)	Interpretable model baseline

6.7 Models

Model	Imbalance handling	Notes
Logistic Regression	`class_weight="balanced"`, `max_iter=2000`	Baseline
Random Forest	`class_weight="balanced_subsample"`, `min_samples_leaf=3`	Slow in CV; keep ≤ 300 trees
XGBoost	`scale_pos_weight = neg/pos`, `eval_metric="aucpr"`	Primary candidate

Matched settings rule: run every model with identical hyperparameters across the with/without-PageValues settings (use the same tree counts), so the ablation is clean.

6.8 Tuning (XGBoost only; RF tuning optional)

```
xgb_space = {
  "clf__n_estimators": randint(150, 500),
  "clf__learning_rate": uniform(0.01, 0.1),
  "clf__max_depth": randint(2, 6),
  "clf__min_child_weight": randint(1, 10),
  "clf__subsample": uniform(0.6, 0.4),
  "clf__colsample_bytree": uniform(0.5, 0.5),
}
RandomizedSearchCV(pipe, xgb_space, n_iter=25, cv=cv,
                   scoring="average_precision", random_state=SEED, n_jobs
```

Run once with PageValues and once without. Treat gains < ~0.01 PR-AUC as "no meaningful gain"; the best-of-search score is optimistically biased.

6.9 OOF Predictions and Threshold Selection

```
oof = cross_val_predict(best_estimator, X_train, y_train, cv=cv, method="
thresholds = np.linspace(0.02, 0.98, 97)
# cost(t) = FP*1 + FN*R   for R in {3, 5, 10}
```

Report per R: best threshold, precision, recall, flagged %, cost relative to the cheaper naive baseline. Fix the primary R (e.g., 5) and threshold **before** the test run and write it down.

6.10 One-Shot Test Evaluation

- PR-AUC and ROC-AUC with bootstrap 95% CI (1,000 resamples; label as test-sampling only).
- Fixed OOF thresholds applied to test: precision, recall, flagged %, costs vs do-nothing and flag-all.
- The `PageValues > 0` rule on test.
- Do not change the model or threshold after seeing these numbers.

6.11 Ablations

Ablation	How	Reports
PageValues	Same tuned XGB with/without, same CV	PR-AUC gap, lift over no-skill
Month	Retrain without Month (CV, paired folds)	Gap per fold; caveat: tuned params favor the with-Month side
Technical ID columns	Drop Browser, OS, Region, TrafficType (CV)	Gap vs fold noise
BounceRates / ExitRates	Drop each for LR	Expected: no measurable change

6.12 Interpretation

1. **Permutation importance** on test (original columns, `scoring="average_precision"`, 5 to 10 repeats).
2. **SHAP (TreeExplainer)** on test; beeswarm + bar.
3. **Grouped SHAP:** sum signed SHAP across one-hot columns of the same original column per session, then mean absolute value.
4. **Binned dependence tables** (text-readable): conversion and mean SHAP per bin for ProductRelated_Duration and ExitRates; mean SHAP per month with n.
5. Compare methods; **report disagreements instead of choosing the flattering one.**

6.13 Lift and Ranking

- Decile table on test: n, buyers, conversion, lift, cumulative recall.
- Compare cumulative recall vs best single feature (ExitRates only) and random.
- Report framing: "better at ruling out than ruling in".

6.14 Artifacts to Save

`xgb_no_pagevalues.joblib`, `xgb_with_pagevalues.joblib`, `app_meta.json`, figures (PR curves, cumulative recall, SHAP beeswarm, cost curves, EDA plots), `requirements.txt` from `pip freeze`.

---

7. Evaluation Protocol Summary

Item	Rule
Headline	PR-AUC (CV and test)
Secondary	ROC-AUC (flatters on imbalanced data), precision/recall/F1 at the fixed threshold
Never headline	Accuracy
CV comparison	Paired fold-by-fold; call a gap real only if consistent across folds; avoid "significant" with 5 folds
Noise floor	Fold std (~~0.03) and bootstrap CI width (~~±0.05)
Cost evidence	Always against the cheaper naive baseline, with R sensitivity

---

8. Reference Results (from the reference implementation; use as sanity targets)

Item	Value
Rows after dedup	12,205 (125 removed); positive rate 15.6%
Split	9,764 train / 2,441 test
Majority accuracy	0.844
Rule `PageValues>0` (test)	P 0.571, R 0.812, F1 0.670
CV PR-AUC with PageValues	LR 0.664, RF 0.734, XGB 0.749 (tuned XGB 0.752)
CV PR-AUC without PageValues	LR 0.330, RF 0.384, XGB 0.372 (tuned XGB 0.382)
Test PR-AUC (tuned XGB)	0.758 with / 0.376 without PageValues

Item	Value
Single-feature PR-AUC (train)	ExitRates 0.259
Top 30% of sessions (no PV model)	~65% of buyers; bottom 40% ≈ 6%
Month ablation	~0.04 CV PR-AUC
ID columns ablation	~0.007 CV PR-AUC (negligible)

If your numbers differ meaningfully, investigate before writing the report.

---

9. Extra Features (with acceptance criteria)

Tier A: Rigor upgrades (about 1 hour each)

Feature	Implementation	Acceptance
Matched-settings comparison	Same hyperparameters for RF/XGB across settings; print per-fold PR-AUC	Single table, identical configs stated
Learning curve	`learning_curve` on the tuned no-PV model, sizes 25/50/75/100%	If CV PR-AUC at 75% vs 100% differs < ~0.02, state "more rows unlikely to help"
SMOTE-in-CV	`imblearn.pipeline.Pipeline([prep, SMOTE, clf])` ; compare with class weights on same folds	Reported with per-fold scores; SMOTE never applied before split
Calibration	`CalibratedClassifierCV` (isotonic or sigmoid) fit inside CV; reliability curve + Brier score	Calibrated probabilities allowed in the app only if reliability curve is near diagonal
Bootstrap CIs on decile lift	Resample test set	Lift table with intervals
UCI metric definitions	Read dataset description; cite	One cited paragraph in limitations

Tier B: Product features

Feature	Implementation	Acceptance
Streamlit app	See Section 10	Both sanity sessions land in the expected bands
K-means segmentation	Scale log1p engagement features (durations, page counts, rates; exclude PageValues and target); choose k via silhouette + elbow (k = 3 to 6); profile each cluster (size, conversion, typical behavior); name clusters	Clusters have clearly different conversion rates and interpretable names; report silhouette honestly (low silhouette is common)
Cost dashboard	Interactive slider for R in the app or notebook (ipywidgets); shows flagged %, recall, saving vs best naive baseline	Matches the static table at R = 3, 5, 10
Score bands	High/Medium/Low from train-OOF score quantiles (70th and 40th percentiles); band conversion rates from test	Band stats printed and shown in the app

Tier C: Modeling depth (only if core is finished)

Feature	Implementation	Acceptance
Nested CV	Outer 5-fold, inner RandomizedSearch	Reports an unbiased tuned PR-AUC; compare with the optimistic best-of-search value
Repeated CV + paired test	`RepeatedStratifiedKFold` (5×3); paired t-test or Wilcoxon on fold scores for LR vs RF vs XGB	Reports p-values with the caveat that folds overlap
Monotonic constraints	XGB `monotone_constraints` (e.g., ExitRates decreasing, ProductRelated_Duration increasing)	Compare PR-AUC; state whether constraints cost anything
Ensemble check	Average RF and XGB probabilities (rank-average)	Keep only if gain exceeds fold noise
Feature engineering	total_pages, total_duration, avg_time_per_page, product_page_share	Keep only if CV gain exceeds noise; otherwise report "no gain"

Tier D: Event-level rebuild (weeks; only if the course rewards scope)

- **Data:** REES46 multi-category store events, RetailRocket, or YOOCHOOSE (verify availability and licenses on Kaggle).
- **Task:** predict purchase using only the **first N events** of a session (N = 3, 5, 10) so predictions are genuinely early.
- **Features:** event counts by type (view/cart), distinct categories/products, time since session start, price statistics, hour/day.
- **Evaluation:** time-based split (train on earlier days, test on later days); PR-AUC vs N curve.
- **Why it matters:** removes the PageValues problem by design and enables true time validation.
- **Risk:** large data (memory), session definition choices, a full pipeline rebuild. Do not start before the core report is complete.

---

10. Streamlit App Specification

10.1 Behavior

- Inputs: product pages and time, admin/info pages and time, ExitRate, BounceRate, Month (10 observed), VisitorType, Weekend, SpecialDay, technical IDs in a collapsed expander.
- Output: **intent band (High/Medium/Low)** plus the held-out conversion rate of that band. **No "probability" shown** unless a calibration step passed (Tier A).
- Footer states limits: PageValues excluded, Month limited to observed months, band rates from a single noisy test set, associations not causes, and that ExitRate/BounceRate are session-level metrics (the tool scores a *finished* session).

10.2 Files

`app.py`, `xgb_no_pagevalues.joblib`, `app_meta.json` (cutoffs, band stats, month list, category options/defaults).

10.3 Export cell (Colab)

```
cut_hi, cut_mid = np.quantile(oof["no PageValues"], [0.70, 0.40])
# compute band stats on test, dump JSON with cutoffs, months, cat_options
```

10.4 Acceptance tests

1. Near-bounce (ExitRate 0.15, 2 product pages, 20 s) → **Low**.
2. Engaged (ExitRate 0.005, 40 pages, 2,000 s, Nov) → **High**.
3. Changing only Month Mar ↔ Nov moves the score.

4. Library versions in the app environment match training versions.

---

11. Repository Structure

```
project/
|-- data/
|   |-- raw/ , processed/shoppers_clean.csv
|-- notebooks/
|   |-- 01_eda.ipynb
|   |-- 02_modeling.ipynb
|   |-- 03_evaluation_interpretation.ipynb
|-- src/              (preprocessing.py, evaluation.py, thresholds.py)
|-- models/           (*.joblib, app_meta.json)
|-- reports/
|   |-- figures/
|   |-- report.pdf
|-- app/              (app.py + model files)
|-- requirements.txt
|-- README.md
```

---

12. Deliverables

Deliverable	Contents
Notebook(s)	Sectioned, markdown headings, a takeaway under each plot, runs top-to-bottom after restart
Report (8 to 12 pages)	See Section 13
README	Problem, dataset link, setup, run order, results summary, limitations
requirements.txt	Pinned versions from `pip freeze`
App (optional)	Section 10
Slides (only if required)	8 to 10 slides from report figures

---

13. Report Outline

1. **Introduction:** task, business use, metric choice, with/without PageValues framing
2. **Data and cleaning:** counts, dedup, split, majority baseline
3. **EDA:** 4 to 5 figures with takeaways (month with n, visitor type, correlation, PageValues split)
4. **Methods:** pipeline, CV, models, imbalance handling, threshold method, pre-registration statement
5. **Model comparison:** matched-settings table, tuning result
6. **PageValues ablation (central finding)**
7. **Threshold and cost analysis:** R assumption, sensitivity, cheaper-baseline comparison
8. **Lift and ranking:** decile table, vs single feature
9. **Interpretation:** permutation + corrected SHAP, binned dependence, Month and ID ablations
10. **(Optional) Segmentation / app / extensions**
11. **Limitations**
12. **Conclusion**

---

14. Timeline (about 2.5 to 3 weeks)

Phase	Days	Output
Setup, clean, EDA	2	EDA notebook
Preprocessing, baselines, LR	1	Baseline table
RF/XGB, matched comparison, tuning	2	Comparison table
OOF threshold, test evaluation	1	Fixed-protocol results
Ablations, SHAP, lift	2	Interpretation tables
Tier A extras (learning curve, SMOTE, calibration)	1	Rigor appendix
Tier B extras (app, segmentation)	2	App + clusters
Report writing	4	Draft + revision
README, restart-and-run-all, buffer	2	Final package

---

15. Risks and Mitigations

Risk	Impact	Mitigation
Reporting accuracy	Misleading	PR-AUC headline; show majority baseline

Risk	Impact	Mitigation
Test-set peeking	Optimistic results	Pre-register choices; one-shot test run
SMOTE before split	Inflated scores	Only inside imblearn pipeline within CV
Calling PageValues "leakage" without proof	Overclaim	Wording: "tied to purchase path by construction"
Unmatched settings in comparison	Weak table	Matched-settings rule (6.7)
Summing |SHAP| across dummies	Wrong ranking	Signed-sum grouping (6.12)
Scores shown as probabilities	Misleading app	Bands, or calibrate first
Strawman cost baseline	Inflated savings	Compare to cheaper naive baseline
Small buyer count (~380 in test)	Wide CIs	Report CIs; avoid decile-level claims
Scope creep	Unfinished report	Tier B/C/D only after core + report draft
Version mismatch (app)	Broken or silently wrong model	Pin identical versions

---

16. Limitations (include in report)

- No timestamps: no time-based validation; Jan/Apr unseen; Month encodes base rates of observed months only.
- PageValues/ExitRates/BounceRates are GA page-level metrics; live use needs past-traffic-only computation (verify against UCI description).
- Single site, single year; limited generalization.
- Cost ratio R is assumed, not measured.
- Session aggregates lose page-order information.
- Scores are not calibrated probabilities (unless Tier A calibration done).
- Bootstrap CIs cover test-sampling uncertainty only.
- Associations, not causal effects.
- List any skipped extension explicitly (e.g., SMOTE, calibration, RF tuning).

---

17. Viva Preparation (write your own one-line answers)

1. Why PR-AUC, not accuracy?
2. Why remove PageValues? Is that unfair to the model?
3. Where does R come from, and what real numbers would replace it?
4. Why choose thresholds on OOF, not test?
5. Why does Month help, and would you trust it for January?
6. Why do SHAP and permutation importance differ, and what was wrong with naive SHAP grouping?
7. Is PR-AUC 0.376 good? (Answer with lift over base rate and over the best single feature.)
8. Why didn't you use more data or deep learning? (Learning-curve evidence.)

---

18. Final Acceptance Checklist

- [ ] Dedup before split; counts reported
- [ ] All transforms/resampling inside pipelines
- [ ] Baselines reported (majority, PageValues rule, single-feature, LR)
- [ ] Matched-settings model comparison
- [ ] PageValues ablation with CIs
- [ ] Threshold from OOF; R sensitivity; cheaper-baseline comparison
- [ ] Test evaluated once; pre-registered choices stated
- [ ] Permutation + grouped SHAP; disagreements reported
- [ ] Month and ID-column ablations
- [ ] Decile lift + single-feature comparison
- [ ] Learning curve (or stated as not done)
- [ ] SMOTE/calibration done or listed as not done
- [ ] Limitations specific and honest
- [ ] Restart-and-run-all succeeds; seeds fixed; versions pinned
- [ ] README, requirements, report complete
- [ ] App (if built) passes Section 10.4 tests